

DIGIZAFE — TEAM MEMBER 3 MASTER TECHNICAL NOTES

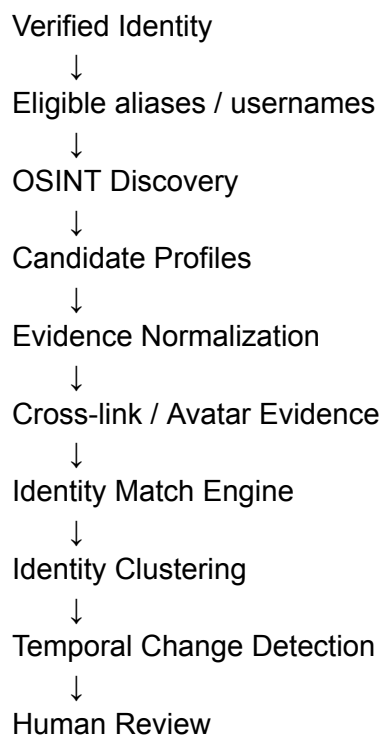
Your role

OSINT & Identity Discovery Engineer

Your main responsibility is:

"I developed the identity discovery and evidence-correlation pipeline that takes verified user identifiers, discovers candidate profiles through OSINT connectors such as Maigret and OSINTgram, normalizes their observations, and feeds evidence into the deterministic Identity Match Engine."

Your area covers the chain:



The project's architecture explicitly places asynchronous OSINT gathering in the Intelligence Layer, separate from the risk engine.

PART I — YOUR ROLE IN THE WHOLE PROJECT

Think of the five major stages:

1. DISCOVER
2. CORRELATE
3. SCORE
4. REMEDIATE
5. PROTECT

Your major contribution:

DISCOVER + CORRELATE

You answer:

"What information about this identity exists publicly, and is this discovered profile plausibly associated with our verified user?"

You do **not** automatically answer:

"This definitely belongs to the user."

That distinction is fundamental to the project's identity architecture.

The Sprint 17 specification explicitly preserves:

discovered $\neq$ owned

similar $\neq$ verified

likely $\neq$ confirmed

algorithmic assessment $\neq$ ownership proof

PART II — THE PROBLEM YOU SOLVE

Traditional OSINT tools can find:

username $\rightarrow$ Instagram

username $\rightarrow$ GitHub

username $\rightarrow$ Reddit

username $\rightarrow$ other platforms

But discovery alone creates a major problem:

False positives.

Suppose the username is:

john_smith

You find:

Instagram → john_smith

GitHub → john_smith

Reddit → john_smith

Does that prove all three belong to your user?

No.

There may be many people using:

john_smith

The project therefore treats OSINT results as:

candidate evidence, not identity proof.

The literature comparison also explicitly identifies Maigret/Sherlock's false-positive problem and says DigiZafe uses them as candidate leads into its Entity Resolution Matrix.

PART III — YOUR COMPLETE PIPELINE

Memorize this:

Verified Identity Anchor



Verified identifiers



User-confirmed aliases

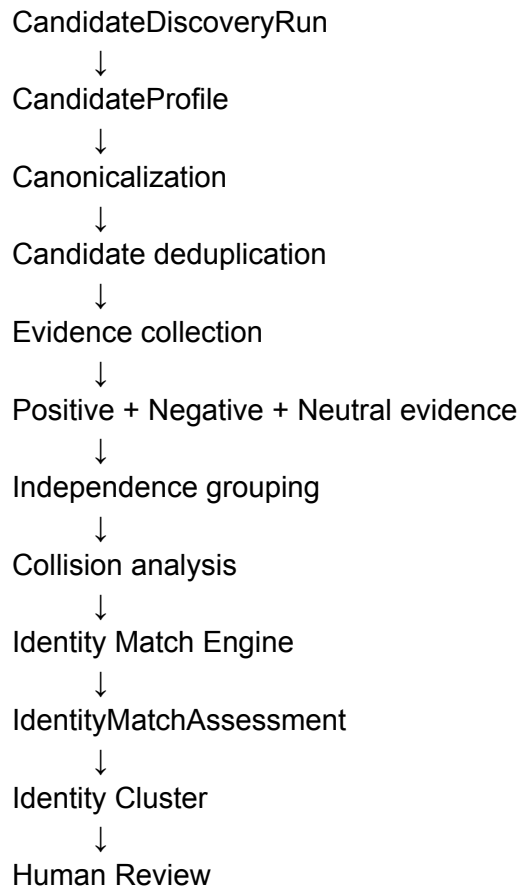


Discovery eligibility policy



Maigret / OSINTgram





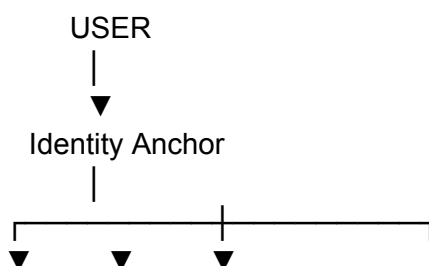
Sprint 17 defines essentially this architecture between candidate discovery and identity confidence.

PART IV — WHAT IS AN IDENTITY ANCHOR?

This is foundational.

An **Identity Anchor** represents the trusted starting point for the user's identity.

Think:



Email Username Alias

The OSINT system should not start from arbitrary strings.

It should start from:

verified identifiers

+

user-confirmed aliases

+

confirmed profiles

Sprint 17 explicitly defines the anchor as the source containing verified identifiers, confirmed aliases and confirmed profiles.

PART V — WHY NOT SEARCH EVERYTHING?

Professor:

"Why don't you search the user's name, email, domain, phone number and everything?"

Because that creates:

- false positives
- privacy problems
- excessive network activity
- uncontrolled discovery
- accidental processing of unrelated data

Sprint 16 therefore defines an explicit discovery eligibility policy.

Eligible inputs normally include:

active verified username-like identifiers

active user-confirmed username/handle aliases

and explicitly excludes automatic use of:

display names

email local-parts

domains
revoked aliases
arbitrary profile URL hints
other users' data

unless a future policy explicitly permits them.

PART VI — INPUT VALIDATION

Before OSINT execution:

Input
↓
Does it belong to authenticated user?
↓
Is it active?
↓
Is type eligible?
↓
Length valid?
↓
No control characters?
↓
Safe username format?
↓
Record anchor version

Only then:

Maigret / OSINTgram

The Sprint 16 policy explicitly requires these checks.

PART VII — MAIGRET

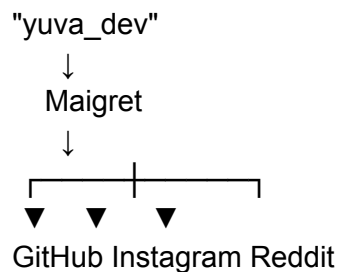
What is Maigret?

Maigret is an OSINT username-discovery tool.

Its purpose in DigiZafe is:

Take an eligible username/handle and discover possible public profiles associated with that username.

Conceptually:



But the results are:

Candidate profiles.

Not confirmed identities.

PART VIII — WHY MAIGRET?

The project comparison identifies Maigret and Sherlock as tools for broad active username candidate discovery, while noting their false-positive limitations. DigiZafe therefore feeds their output into its own identity-resolution process.

So your explanation should be:

"Maigret provides breadth of discovery; DigiZafe's deterministic identity engine provides controlled interpretation."

That's a very strong answer.

PART IX — MAIGRET DOES NOT DECIDE OWNERSHIP

This is one of the most important viva questions.

Professor:

"If Maigret finds the username, does that mean it's the user's account?"

Answer:

"No. Maigret only provides a candidate observation. DigiZafe treats the observation as evidence and evaluates it together with source reliability, independent evidence, collision risk, contradictions and user decisions."

This principle is explicitly built into Sprint 17.

PART X — MAIGRET EXECUTION SAFETY

Maigret isn't simply executed with arbitrary user input.

The system requires:

verified/eligible input
+
consent
+
feature flag
+
bounded execution

Sprint 16 requires:

hard execution timeout
concurrency limit
maximum usernames/run
maximum persisted candidates
maximum raw output

PART XI — FEATURE FLAG

Maigret discovery is controlled by:

FEATURE_MAIGRET_DISCOVERY

Recommended default:

false

until acceptance.

When disabled:

No Maigret execution

No subprocess

No candidate discovery network activity

PART XII — WHY PIN THE MAIGRET VERSION?

Imagine today:

Maigret v0.4.4

Tomorrow upstream changes behavior.

Your output may change.

That damages:

- reproducibility
- testing
- evidence provenance
- debugging

Therefore the project requires pinning the tool/version and recording:

tool name

tool version

installation method

checksum/lockfile

runtime version

known limitations

PART XIII — CONSENT BOUNDARY

This is a major security point.

If the user hasn't granted the required discovery consent:

Discovery request



Consent check



DENIED



403 / canonical denial



NO worker

NO subprocess

NO external request

The Sprint 16 specification explicitly requires zero worker execution and zero external discovery egress when required consent is missing.

PART XIV — CANDIDATE DISCOVERY RUN

Every discovery operation needs durable execution state.

The project uses:

CandidateDiscoveryRun

Conceptually:

CandidateDiscoveryRun

|

|— user_id

|— anchor_id

|— anchor_version

|— source/tool

|— status

|— started_at

|— completed_at

|— input_count
|— candidate_count
|— error_code

Possible states:

queued
running
completed
partially_completed
failed
cancelled
timed_out

PART XV — WHY DO WE NEED A RUN MODEL?

Without a run record:

Maigret
↓
50 profiles

and later someone asks:

"Where did these profiles come from?"

You wouldn't know:

- which username was used
- when discovery occurred
- which tool version was used
- whether the run partially failed
- how many candidates were found

A durable discovery run provides provenance.

PART XVI — CANDIDATE PROFILE

After discovery, raw observations are normalized into:

CandidateProfile

Important conceptual fields include:

user_id
discovery_run_id
anchor_id
anchor_version
platform
profile_url
canonical_profile_url
username_observed
display_name_observed
observation_status
candidate_status
source_tool
source_tool_version
first_observed_at
last_observed_at

PART XVII — CANDIDATE VS OBSERVATION

Very important.

Observation

"Maigret reported this profile."

Candidate

"We normalized that observation into a potential profile entity."

So:

Tool Observation



Candidate Profile

The project explicitly distinguishes these concepts.

PART XVIII — WHY CANONICAL PROFILE URL?

Suppose the same profile appears as:

`https://example.com/user/yuva`

and:

`https://example.com/user/yuva/`

or with:

`?utm_source=abc`

You don't want to create three candidates.

Therefore DigiZafe canonicalizes URLs.

Typical normalization:

- lowercase hostname
- remove default ports
- strip fragments
- normalize trailing slash where safe
- remove tracking parameters
- preserve path case when required

PART XIX — USER-SCOPED DEDUPLICATION

Suppose:

User A → profile X

User B → profile X

You must not create a globally visible identity graph saying:

"Profile X belongs to both."

Instead deduplication is user-scoped:

user_id
+
canonical platform
+
canonical profile URL

defines one candidate identity location within that user's scope.

PART XX — WHY USER-SCOPED?

Because a global identity index could leak:

User B discovered this profile.

to User A.

That's a privacy violation.

Therefore:

Candidate deduplication must not become cross-user identity correlation.

PART XXI — PROVENANCE

Every candidate should be explainable.

For example:

Candidate:
Instagram / yuva123

Discovered by:
Maigret

Run:

#1234

Tool version:

...

Observed:

2026-08-10

The project explicitly requires provenance to be preserved so later evidence processing can understand where a candidate came from.

PART XXII — OSINTGRAM

Maigret is one discovery connector.

The later architecture adds:

OSINTgram

OSINTgram is integrated as another connector for Instagram-oriented discovery.

The important concept isn't simply:

Maigret + OSINTgram

It is:

Multiple independent connectors



Normalized observations



Same identity evidence pipeline

PART XXIII — WHY ADD OSINTGRAM?

Maigret provides broad username discovery.

OSINTgram provides additional Instagram-oriented observations.

So multiple connectors can potentially provide:

username
profile URL
platform
bio
relationships
public observations

But:

More connectors ≠ automatic confirmation.

The Sprint 19 UI guidance explicitly says:

"Discovered by Maigret / OSINTgram" is provenance, not a confidence score.

PART XXIV — THE BIG MISTAKE TO AVOID

Never say:

"Two tools found the account, therefore it's verified."

The project explicitly rejects:

2 tools found this



verified

Instead:

Maigret observation

+

OSINTgram observation



Evidence



Independence analysis



Identity Match Engine

PART XXV — OSINTGRAM ADAPTER

The system creates:

OSINTgramAdapter

Its job is to translate OSINTgram's output into DigiZafe's internal connector contract.

Instead of spreading raw CLI parsing throughout the application:

OSINTgram CLI

↓

Adapter

↓

ConnectorExecutionResult

The documented result structure contains:

status

connector

connector_version

capability

observations

warnings

started_at

completed_at

error_code

PART XXVI — WHY AN ADAPTER?

Without an adapter:

Business logic

↓

OSINTgram CLI output

Now your whole application depends on OSINTgram's output format.

With an adapter:

OSINTgram

↓

OSINTgramAdapter



Standard DigiZafe contract



Business logic

If OSINTgram changes its output, ideally only the adapter needs modification.

PART XXVII — CONNECTOR OUTPUT IS UNTRUSTED

Very important.

OSINT connector output must be treated as:

Untrusted external input.

Therefore the adapter validates:

schema

types

length

URLs

unexpected fields

malformed records

One malformed observation should not necessarily crash an entire run if safe partial processing is possible.

PART XXVIII — PARTIAL RESULTS

Suppose OSINTgram finds:

Profile A

Profile B

Profile C

and then crashes.

You should not say:

FAILED → throw everything away

Instead:

Valid observations



Preserve



Run = partial_result

The project explicitly requires preservation of valid normalized observations while marking the run as partial.

PART XXIX — CONNECTOR FAILURE ≠ NEGATIVE EVIDENCE

This is a **very important viva question**.

Suppose:

OSINTgram unavailable

Does that mean:

"The user has no Instagram profile?"

Absolutely not.

It means:

"We couldn't obtain an observation."

Therefore:

Connector failure



unknown / operational failure

NOT:

Connector failure



negative identity evidence

The project explicitly requires this distinction.

PART XXX — CONNECTOR HEALTH

Connector states include:

available
disabled
not_configured
session_expired
temporarily_rate_limited
degraded
unavailable

This lets the UI explain:

"Instagram connector unavailable"

instead of falsely implying:

"No Instagram account found."

PART XXXI — RATE LIMITING

OSINT connectors can make many external requests.

Therefore the project requires bounded limits such as:

max runs/time window
max usernames/run
max relationship targets/run
max concurrent connector runs
cooldown after repeated failures

Why?

Without limits:

User

↓

100 usernames

↓

20 platforms each

↓

2000 requests

could cause:

- rate limiting
- resource exhaustion
- connector bans
- unnecessary network traffic

PART XXXII — TEMPORARY WORKSPACE

Each connector run should use an isolated temporary workspace.

Conceptually:

Run 1 → /tmp/run-abc

Run 2 → /tmp/run-def

and clean it after:

success

failure

timeout

worker exception

The project explicitly requires cleanup and prohibits user-controlled persistent output paths.

PART XXXIII — SPRINT 17:

IDENTITY MATCH ENGINE

This is your **core reasoning layer**.

The problem:

CandidateProfile

exists.

Now we ask:

"How likely is this candidate associated with the verified user?"

The answer must be:

deterministic + explainable.

Sprint 17 explicitly says the engine must be deterministic and that an LLM must not be the authoritative scorer.

PART XXXIV — WHY NOT ML FOR IDENTITY MATCHING?

Professor:

"You already use machine learning in DigiZafe. Why not use ML here?"

Answer:

"Because identity attribution is a high-impact decision where we need to explain exactly why a profile was linked to a user. A deterministic evidence-based engine gives us reproducibility, auditability and explicit handling of contradictions and uncertainty. ML is used elsewhere for residual risk scoring, but it is not the authoritative identity matcher."

The project explicitly distinguishes the deterministic identity engine from the ML risk scorer.

PART XXXV — EVIDENCE TYPES

The identity engine considers:

Positive evidence

Evidence supporting a match.

Example:

Exact username match

Negative evidence

Evidence against a match.

Example:

User explicitly dismissed profile

Neutral / unknown

No meaningful conclusion.

Example:

Connector unavailable

Sprint 17 explicitly defines positive, negative and neutral/unknown evidence.

PART XXXVI — EVIDENCE STRENGTH

The engine categorizes evidence by strength.

The project's documented scoring model uses:

Strong = 70

Moderate = 40

Weak = 20

But don't describe these numbers as probabilities.

They are:

match-score contributions.

PART XXXVII — WHY NOT CALL 70 = 70%?

Because:

score $\neq$ calibrated probability

The project explicitly warns against exposing the score as a probability unless it has actually been calibrated.

So say:

"The score is an internal bounded match score, not a probability."

PART XXXVIII — INDEPENDENCE GROUPS

This is one of the most important concepts.

Imagine:

Broker A $\rightarrow$ username = yuva123

Broker B $\rightarrow$ username = yuva123

Broker C $\rightarrow$ username = yuva123

Maybe all three copied the same original source.

Then:

3 observations

do **not** necessarily mean:

3 independent pieces of evidence.

The engine therefore groups related evidence.

PART XXXIX — WHY NO DOUBLE COUNTING?

Suppose one underlying fact is:

username = yuva123

It appears in:

profile URL

profile username field

Maigret output

You shouldn't score:

+20

+20

+20

and pretend that is three independent facts.

The Sprint 17 rule is:

one underlying fact

→ one bounded contribution

PART XL — IDENTITY MATCH STATUSES

The engine can produce:

likely_match

possible_match

insufficient_evidence

unlikely_match

conflicting_evidence

These are extremely important.

PART XLI — LIKELY MATCH

The project documents the core rule:

```
Score >= 70
AND
(
  >= 2 independent evidence groups
  OR
  >= 1 authoritative group
)
```

then:

likely_match

This rule is documented in the project's identity correlation material.

PART XLII — EXAMPLE

Suppose:

Exact username match → Strong = 70

But username is:

john_smith

and highly collision-prone.

You **cannot automatically conclude**:

likely_match

because username collision risk matters.

That's why the engine has:

Collision policy.

PART XLIII — USERNAME COLLISION

Suppose:

yuva_2004

is distinctive.

Versus:

john

which is common.

An exact username match is much less informative for:

john

than:

yuva_2004

The project therefore considers:

- username length
- character diversity
- common-name patterns
- numeric suffixes
- candidate-set frequency
- number of profiles using username

for collision heuristics.

PART XLIV — DON'T INVENT GLOBAL PROBABILITIES

Never say:

"This username has a 0.003% chance of collision."

Unless you have a validated dataset.

The project explicitly says to use deterministic local collision heuristics when no reliable global frequency dataset exists.

PART XLV — NEGATIVE EVIDENCE

This is just as important as positive evidence.

Examples:

User dismissed candidate

or:

Confirmed conflicting profile

or:

Known incompatible identity reference

PART XLVI — WHAT IS NOT NEGATIVE EVIDENCE?

These should **not automatically** mean the candidate isn't the user:

No avatar

No bio

Private profile

No display name

Why?

Because:

absence of information

doesn't automatically mean:

evidence against identity.

The project explicitly prohibits treating these as negative evidence without a documented rule.

PART XLVII — CONTRADICTIONS

Suppose:

Strong username evidence

+

User explicitly says "not mine"

Don't simply average:

+70

-70

= 0

The system should surface:

conflicting_evidence

The project specifically says contradictions should be represented explicitly rather than averaged away.

PART XLVIII — STALENESS

Evidence changes over time.

For example:

Candidate found 8 months ago

Maybe:

profile deleted
alias revoked
anchor changed
URL changed

Therefore evidence can have states such as:

active
stale
superseded
revoked

PART XLIX — ANCHOR VERSION

Every assessment should remember:

anchor_version

Why?

Suppose:

2026:
User aliases = A, B

Later:

2027:
Alias B revoked

An old assessment based on B should not silently be treated as current.

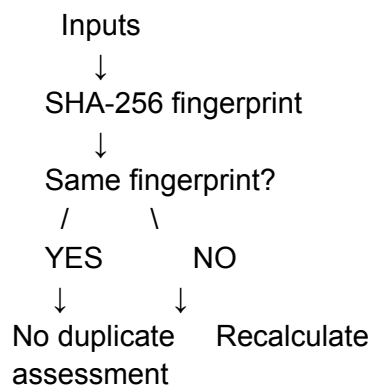
The project requires anchor-version awareness for reproducibility.

PART L — ASSESSMENT FINGERPRINT

The system can calculate an input fingerprint based on:

candidate
+
anchor version
+
material evidence
+
engine version
+
policy version

Conceptually:



This prevents unnecessary repeated calculations.

PART LI — SPRINT 18:

CROSS-LINK EVIDENCE

Now the system becomes more sophisticated.

Instead of relying only on:

username

we can use bounded public cross-links.

Example:

Profile A
↓
public link
↓
Profile B

This creates additional evidence.

But:

Cross-link ≠ ownership proof.

That is one of Sprint 18's explicit invariants.

PART LII — AVATAR SIMILARITY

Sprint 18 introduces:

Non-biometric avatar similarity.

Allowed methods include:

SHA-256 exact-byte hash
pHash
dHash
aHash
bounded image metadata

PART LIII — VERY IMPORTANT:

THIS IS NOT FACE RECOGNITION

The project explicitly prohibits:

FaceNet
ArcFace
DeepFace
facial embeddings
facial landmarks
biometric templates
face recognition

The system compares:

images

not:

faces/people.

PART LIV — HOW TO EXPLAIN AVATAR SIMILARITY

Suppose:

Profile A avatar



pHash



ABC123

and:

Profile B avatar



pHash



ABC127

The hashes are close.

You can say:

"The profiles appear to reuse the same or visually similar avatar image."

You cannot say:

"The faces belong to the same person."

That distinction is explicitly mandated by Sprint 18.

PART LV — WHY PHASH?

A normal SHA-256 hash changes completely if even one byte changes.

Example:

Image A
SHA-256 → ABC123...

Image B
SHA-256 → F9821...

Even if visually almost identical.

Perceptual hashing instead attempts to encode visual characteristics.

So:

Visually similar images
↓
similar perceptual hashes

This is useful for detecting possible image reuse.

PART LVI — SPRINT 18 CLUSTERING

The system can create:

Identity clusters.

Imagine:

Candidate A
Candidate B
Candidate C

Evidence:

A ↔ B

B ↔ C

may indicate a related cluster.

But:

Cluster membership ≠ confirmed identity.

That is another explicit Sprint 18 invariant.

PART LVII — WHY CLUSTERING?

Without clustering:

50 candidate profiles

can be difficult to interpret.

With conservative clustering:

Cluster 1

├── Profile A
├── Profile B
└── Profile C

Cluster 2

├── Profile D
└── Profile E

The investigator can reason about related evidence more efficiently.

PART LVIII — TRANSITIVE CONFLICT

This is an advanced viva question.

Suppose:

A supports B
B supports C

You might incorrectly conclude:

A supports C

But suppose:

A conflicts with C

Then:

A + B + C

cannot automatically become one supported identity cluster.

Sprint 20 explicitly preserves the Sprint 18 transitive-conflict safety rule.

PART LIX — FEATURE FLAGS

Sprint 18 uses bounded flags:

```
FEATURE_IDENTITY_CROSS_LINKS=false  
FEATURE_AVATAR_SIMILARITY=false  
FEATURE_IDENTITY_CLUSTERING=false
```

Safe defaults are:

false

Why?

Because new evidence mechanisms should not silently activate in production.

PART LX — SPRINT 19:

OSINTGRAM INTEGRATION

Now the system has:

Maigret
+
OSINTgram

The architecture doesn't make OSINTgram a separate identity engine.

Instead:

OSINTgram
↓
Adapter
↓
Normalized observation
↓
Candidate provenance
↓
Existing evidence pipeline

PART LXI — CANDIDATE PROVENANCE

Suppose:

Candidate X

was discovered by:

Maigret
OSINTgram

The system stores:

Candidate X
├─ provenance: Maigret
└─ provenance: OSINTgram

But this is **not**:

confidence = 2

It is:

"Two sources observed this candidate."

Sprint 19 explicitly requires this distinction.

PART LXII — WHY PROVENANCE MATTERS

Later the identity engine can ask:

Where did this fact originate?

rather than merely:

What is the fact?

This is essential for:

- explainability
 - auditing
 - evidence independence
 - debugging
 - revalidation
 - deletion/export
-

PART LXIII — SPRINT 20:

CONNECTOR ORCHESTRATION

Before Sprint 20, you can think:

Run Maigret
Run OSINTgram
Run avatar analysis

But that becomes inefficient.

Sprint 20 introduces:

Connector Orchestration

The platform decides:

Which connector?

When?

Why?

Should it run?

Is it healthy?

Is the result fresh?

Is the user within budget?

PART LXIV — CONNECTOR REGISTRY

Important services include:

ConnectorRegistry

ConnectorOrchestrationService

ConnectorEligibilityPolicy

ConnectorBudgetService

ConnectorHealthService

EvidenceFreshnessService

CanonicalFactService

IdentityReassessmentCoordinator

PART LXV — WHY ORCHESTRATION?

Suppose you have:

Maigret

OSINTgram

Avatar Similarity

Cross-link enrichment

You don't want:

EVERY observation



EVERY connector



EVERY time

Instead:

Observation



Policy



Which connector is useful?



Execute only necessary work

PART LXVI — EXECUTION ORDER

The project suggests a deterministic order such as:

1. lower-cost / passive connector
2. source-specific connector
3. enrichment only for newly material observations

This reduces unnecessary work.

PART LXVII — PARALLEL CONNECTORS

Some independent connectors can execute concurrently.

But concurrency is bounded by:

per-user concurrency
per-connector concurrency
queue capacity
budget
health

So:

Concurrent doesn't mean unlimited.

PART LXVIII — QUEUE SEPARATION

The project deliberately keeps workloads logically separated:

Maigret

↓

discovery/scans queue

OSINTgram

↓

osint_connectors queue

Avatar / cross-link

↓

identity_enrichment queue

The orchestrator coordinates them without collapsing all workloads into one queue.

PART LXIX — ORCHESTRATION STATE MACHINE

Memorize:

PLANNED

↓

QUEUED

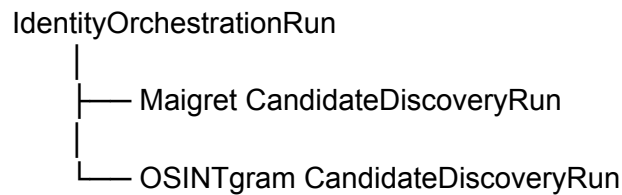
↓

RUNNING

- |— all complete → COMPLETED
- |— some succeed/fail → PARTIAL_RESULT
- |— fatal failure → FAILED

PART LXX — CHILD RUNS

One orchestration run can contain:



This hierarchy is explicitly defined in Sprint 20.

This is a very good architecture question.

PART LXXI — CIRCUIT BREAKER

Suppose OSINTgram repeatedly fails:

failure
failure
failure
failure

Don't continue:

retry
retry
retry
retry

Instead:

CLOSED
↓
too many failures
↓
OPEN
↓
don't execute connector
↓
later
↓

HALF_OPEN

↓

test recovery

The project specifies a bounded connector circuit breaker with closed/open/half-open states.

PART LXXII — WHY CIRCUIT BREAKER?

It protects:

- system resources
- connector sessions
- external services
- queues

and prevents:

retry storms.

PART LXXIII — FRESHNESS

Suppose you already discovered:

Instagram profile

10 minutes ago.

Do you need to query it again immediately?

Probably not.

Sprint 20 introduces evidence freshness policies.

Conceptually:

Observation

↓

Fresh?

├ YES → reuse
└ NO → re-observe

This reduces unnecessary connector work.

PART LXXIV — MATERIAL CHANGE

Not every new observation requires identity reassessment.

Material:

- new canonical fact
- fact superseded
- fact expires
- new contradiction
- user confirmation
- user dismissal
- alias revoked
- confirmed profile revoked
- policy change

Non-material:

- same fact observed again
- duplicate provenance
- same raw result

PART LXXV — WHY MATERIALITY?

Imagine the same fact is observed 100 times.

Without materiality:

- 100 observations
- ↓
- 100 identity assessments

That's wasteful.

Instead:

- same fact
- ↓
- provenance update

↓
no unnecessary reassessment

PART LXXVI — INCREMENTAL REASSESSMENT

Suppose:

100 candidate profiles

and only:

Candidate #37 changed.

You don't want:

Recalculate all 100

Instead:

Candidate #37 changed

↓
Find affected assessment

↓
Recalculate affected candidate

↓
Find affected clusters

↓
Rebuild only affected scope

The Sprint 20 `IdentityReassessmentCoordinator` is designed for exactly this.

PART LXXVII — ASSESSMENT FINGERPRINT

Suppose:

Candidate 37
Anchor version 5
Evidence set ABC
Engine v2
Policy v3

produces:

fingerprint = XYZ

If exactly the same input comes again:

XYZ

there's no reason to create duplicate assessment history.

Sprint 20 specifies this behavior.

PART LXXVIII — SPRINT 21:

TEMPORAL IDENTITY EVIDENCE

This is the next evolution.

Previously:

Candidate exists

Now we ask:

"What changed over time?"

For example:

Monday:
Bio = Student

Wednesday:
Bio = Developer

Friday:

External link changed

DigiZafe needs to represent these changes.

PART LXXIX — IDENTITY CHANGE EVENT

Sprint 21 introduces:

IdentityChangeEvent

The project defines this as the historical representation of meaningful identity-related changes.

Conceptually:

Old state



New state



Change detection



IdentityChangeEvent

PART LXXX — WHY TEMPORAL EVIDENCE?

A single snapshot says:

Current profile = X

A timeline says:

Profile changed from A → B → C

This can provide much better context.

But:

Historical change does not automatically mean malicious activity.

PART LXXXI — OBSERVATION TIME VS DETECTION TIME

Very important.

Suppose:

Observation happened:
Monday 10 AM

but DigiZafe processed it:

Tuesday 2 PM

These are different.

The system therefore distinguishes:

observed_at
detected_at

and potentially:

source_reported_at

The Sprint 21 specification explicitly requires this separation.

PART LXXXII — OUT-OF-ORDER EVENTS

Suppose:

New observation arrives first

Old observation arrives later

Database insertion order would be wrong.

Therefore:

Connector A
old observation

Connector B
new observation

results arrive:
 $B \rightarrow A$

The system should use:

observation time
run lineage
policy

rather than simply:

database insertion order

PART LXXXIII — ABSENCE IS NOT DISAPPEARANCE

Suppose:

Today's OSINT scan:
No Instagram profile found

Does that mean:

"The account was deleted"?

No.

The connector could have:

- failed
- been rate-limited
- incomplete results
- lost session
- changed behavior

Therefore the project explicitly separates:

absence suspected

from:

confirmed disappearance

and connector failure must not create disappearance.

PART LXXXIV — REVIEW QUEUE

Not every change should automatically modify identity state.

Instead, important changes can create:

IdentityReviewItem

This creates:

Change detected



Is it material?



YES



Review Queue



Human decision

The Sprint 21 definition requires review-item deduplication, deterministic grouping where enabled and safe concurrent resolution.

PART LXXXV — REVIEW OPTIONS

The documented initial resolutions include:

ACKNOWLEDGED
THIS_IS_STILL_MINE
THIS_IS_NOT_MINE
EXPECTED_CHANGE
REQUEST_REVALIDATION
DISMISS_ALERT

PART LXXXVI — IMPORTANT:

REVIEW QUEUE DOES NOT BECOME THE SOURCE OF TRUTH

Suppose user clicks:

THIS_IS_STILL_MINE

The review queue should **not directly rewrite every identity table**.

Instead it invokes canonical domain services such as:

ConfirmedProfileReference
CandidateProfile confirmation
IdentityAlias revocation

The Sprint 21 design explicitly says the review queue coordinates decisions but does not replace canonical identity models.

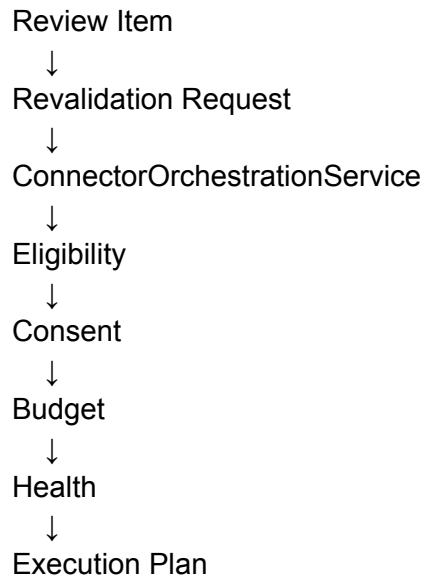
PART LXXXVII — REVALIDATION

Suppose the user clicks:

REQUEST_REVALIDATION

The review system should not directly execute Maigret or OSINTgram.

Instead:



This is a very strong architecture principle.

PART LXXXVIII — AUTOMATIC REVALIDATION

The project allows an optional:

`FEATURE_AUTOMATIC_REVALIDATION=true`

but only approved change types should trigger it.

Examples:

stale high-materiality fact
absence suspected on confirmed profile
high-priority contradiction

and even then:

consent

budget
health
availability
cooldown

must still be enforced.

PART LXXXIX — REVALIDATION COOLDOWN

Without cooldown:

change
↓
revalidate
↓
change
↓
revalidate
↓
change
↓
revalidate

could create a loop.

The documented recommended default is:

24 hours

for the same fact's automatic revalidation.

PART XC — REVIEW GROUPING

Suppose one profile changes:

Bio
Avatar

External link

within a short period.

You don't want:

Alert 1

Alert 2

Alert 3

necessarily.

Instead the project proposes a:

30-minute change burst window

for review grouping.

PART XCI — TIMELINE VS REVIEW QUEUE

Memorize this distinction.

Timeline

All meaningful historical changes.

Review Queue

Only changes requiring user attention.

Therefore:

Timeline



many events

Review Queue



important subset

PART XCII — EVENT FINGERPRINT

To prevent duplicate change events, Sprint 21 proposes a deterministic SHA-256 fingerprint based on:

user scope
canonical fact key
change type
previous state fingerprint
new state fingerprint
change policy version

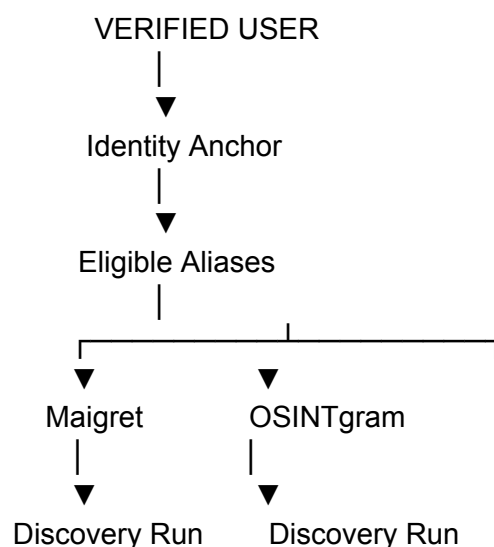
So:

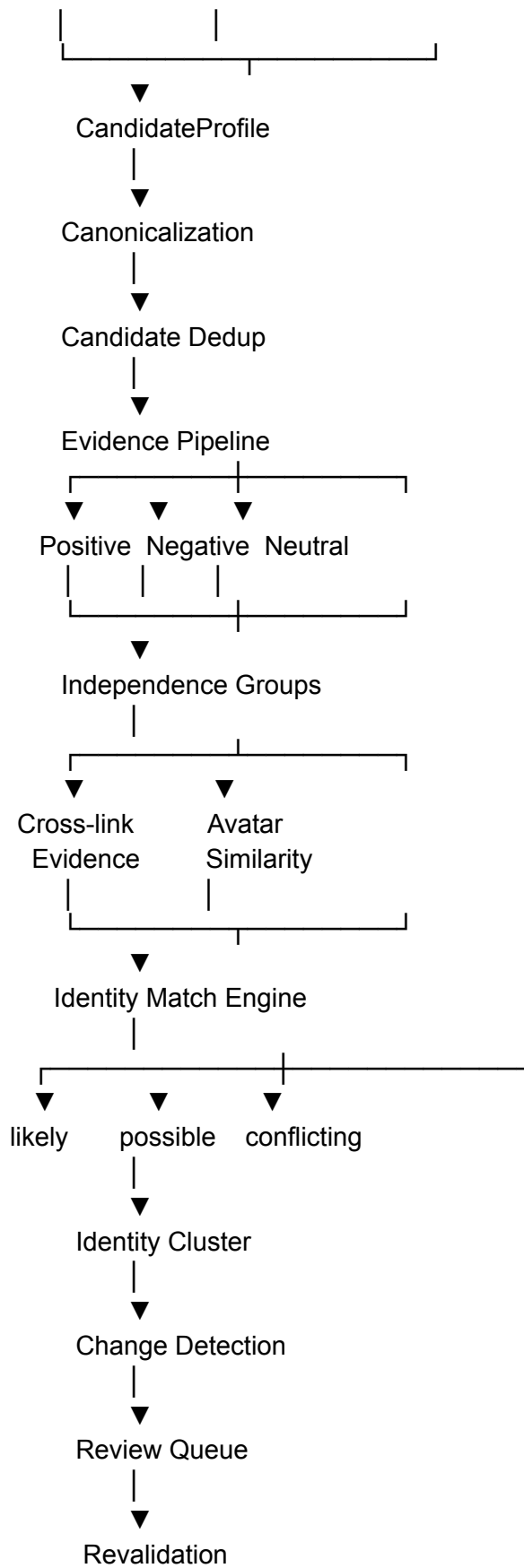
same change
+
same policy
=
same fingerprint

and duplicate processing can be prevented.

PART XCIII — COMPLETE MEMBER 3 ARCHITECTURE

Memorize this diagram:





PART XCIV — YOUR MOST IMPORTANT FILES

MUST KNOW

`identity_match_engine.py`

Know:

- evidence scoring
- independence
- collision policy
- contradiction handling
- assessment status
- explainability
- fingerprints

The project identifies this as the authoritative deterministic identity correlation component.

Maigret Adapter / discovery task

Know:

- eligible inputs
 - consent
 - feature flag
 - subprocess execution
 - timeout
 - output limits
 - candidate creation
 - provenance
-

`OSINTgramAdapter`

Know:

- connector contract
- parsing
- provenance
- session handling
- partial results

- failure states

The project explicitly specifies the adapter contract and version pinning.

CandidateProfile

Know:

candidate

≠

confirmed identity

SHOULD KNOW

CandidateDiscoveryRun

CandidateProvenanceObservation

IdentityMatchAssessment

IdentityCluster

IdentityOrchestrationRun

ConnectorExecutionPlanItem

IdentityChangeEvent

IdentityReviewItem

PART XCV — 30 QUESTIONS YOU MUST MASTER

Q1. What is your contribution?

"I worked on DigiZafe's OSINT discovery and identity-correlation pipeline. I integrated candidate discovery through Maigret and OSINTgram, normalized and deduplicated candidate profiles, preserved provenance, and fed evidence into the deterministic Identity Match Engine."

Q2. What does Maigret do?

It discovers possible public profiles associated with an eligible username/handle.

Q3. Does Maigret prove identity?

No. It produces candidate observations.

Q4. Why not directly trust Maigret?

Because username collisions and false positives are possible.

Q5. What is CandidateProfile?

A normalized potential profile derived from one or more OSINT observations.

Q6. Why canonicalize URLs?

To prevent equivalent URLs from creating duplicate candidates.

Q7. Why user-scoped deduplication?

To prevent cross-user identity leakage and ensure candidate ownership remains scoped to the authenticated user.

Q8. Why store provenance?

To know which connector produced each observation and support explainability and auditing.

Q9. What is OSINTgram?

An additional OSINT connector integrated through an adapter that produces normalized observations.

Q10. Why use an adapter?

To isolate connector-specific output parsing from the rest of the application's business logic.

Q11. What happens if OSINTgram fails?

The failure is classified as an operational failure; it does not become negative identity evidence.

Q12. What happens if OSINTgram partially succeeds?

Valid observations are preserved and the run is marked as `partial_result`.

Q13. Why pin connector versions?

To make discovery reproducible and prevent unexpected upstream changes.

Q14. Why have a consent boundary?

OSINT discovery processes external data and therefore should execute only after the required user consent and authorization checks.

Q15. Why feature flags?

To prevent optional discovery capabilities from silently executing when disabled.

Q16. Why deterministic identity matching?

Because identity attribution needs reproducibility and explainability.

Q17. Why not an LLM?

An LLM is not deterministic enough to serve as the authoritative identity scorer.

Q18. What is an independence group?

A mechanism for preventing correlated or duplicated observations from being treated as independent evidence.

Q19. Why is double counting dangerous?

Because multiple copies of the same underlying fact could artificially inflate the match score.

Q20. What is collision risk?

The likelihood that a username or identifier could plausibly belong to multiple unrelated people.

Q21. What is negative evidence?

Supported evidence that argues against the candidate belonging to the user, such as an explicit user dismissal or documented contradiction.

Q22. What is conflicting evidence?

A situation where meaningful evidence supports and contradicts the same candidate, which should be surfaced rather than averaged away.

Q23. What is avatar similarity?

Non-biometric comparison of images using techniques such as pHash/dHash; it is not facial recognition.

Q24. Does avatar similarity prove two accounts belong to the same person?

No. It only indicates possible reuse or visual similarity of images.

Q25. What is identity clustering?

Conservative grouping of related candidate profiles based on bounded evidence; cluster membership is not identity confirmation.

Q26. What is orchestration?

The policy-driven coordination of multiple connectors, including eligibility, freshness, budget, health, ordering and concurrency.

Q27. Why not run all connectors every time?

It wastes resources, increases external requests and may violate budgets or rate limits.

Q28. What is incremental reassessment?

Recalculating only candidates and clusters affected by material evidence changes rather than rebuilding everything.

Q29. What is temporal evidence?

Evidence tracked across time so the system can identify meaningful changes instead of treating every scan as an isolated snapshot.

Q30. What is the most important limitation?

OSINT discovery depends on external sources whose availability, coverage, rate limits and data quality can change; therefore discovered information is inherently incomplete and must be treated as evidence rather than absolute truth.

The project explicitly lists external OSINT availability, WAF/CAPTCHA restrictions, third-party rate limits and limited adaptive behavior as system limitations.

PART XCVI — 15 HARD PROFESSOR QUESTIONS

1. If two tools find the same profile, is confidence doubled?

No.

If the observations derive from the same underlying fact/source, they may belong to the same independence group.

2. If Maigret finds nothing, does the user have no account?

No.

It means no candidate was observed by that connector under that query/policy.

3. If OSINTgram fails, is that negative evidence?

No.

It's operational uncertainty.

4. Why not search the user's email local part?

Because it can create unintended matches and isn't automatically an eligible discovery input under the defined policy.

5. Why not search display name?

Because display names are highly collision-prone and aren't automatically eligible discovery inputs.

6. What happens if the same profile appears twice?

Canonicalization and user-scoped deduplication prevent uncontrolled duplicate candidates.

7. Why not globally deduplicate profiles?

Because that could expose cross-user identity information.

8. What if a username is extremely common?

Apply collision-aware evidence limits rather than treating exact username equality as strong proof.

9. Why does a contradiction override normal scoring?

Because averaging contradictory evidence can hide an important conflict. The system should explicitly surface `conflicting_evidence`.

10. What if evidence becomes stale?

The assessment can become stale and be recalculated using current eligible evidence.

11. Why preserve historical assessments?

For auditability and reproducibility.

12. Why use fingerprints?

To determine whether the effective evidence/input set has actually changed and avoid duplicate recalculation.

13. Why use a circuit breaker?

To stop repeated connector failures from causing endless retries.

14. Why incremental clustering?

Because rebuilding every identity cluster for every observation is unnecessarily expensive.

15. What happens when an account disappears from a scan?

Do not immediately conclude deletion.

It could be:

connector failure

rate limit

partial result

temporary unavailability

The temporal design explicitly distinguishes absence suspicion from confirmed disappearance.

PART XCVII — YOUR 5-MINUTE CONTRIBUTION EXPLANATION

If the professor asks:

"Explain your contribution to DigiZafe."

Use this:

"My main contribution was the OSINT discovery and identity-correlation layer of DigiZafe. The challenge we identified was that existing OSINT tools can discover usernames and profiles, but a discovered profile cannot automatically be assumed to belong to the authenticated user."

We therefore built a pipeline beginning with a verified Identity Anchor and eligible user-confirmed aliases. Discovery connectors such as Maigret and OSINTgram operate only within the defined consent and eligibility boundaries. Their results are persisted as CandidateDiscoveryRuns and normalized into CandidateProfiles with canonical URLs and provenance information.

The important part is that discovery and ownership are separate concepts. A Maigret or OSINTgram result is only a candidate observation. We then normalize the evidence and pass it through the deterministic Identity Match Engine. The engine considers positive and negative evidence, source reliability, independence groups, username collision risk and contradictions. This prevents repeated or correlated observations from artificially increasing confidence.

Sprint 18 extends this with bounded cross-link evidence, non-biometric avatar similarity using techniques such as pHash and dHash, and conservative identity clustering. We explicitly do not use facial recognition; visual similarity is not treated as identity proof.

Sprint 19 adds OSINTgram through an adapter so its connector-specific output is converted into a stable internal observation contract. Sprint 20 then introduces connector orchestration, which decides which connectors should run based on eligibility, freshness, budget, health and concurrency. Finally, Sprint 21 adds temporal evidence and change detection so we can track meaningful identity changes over time and route important changes to a human review queue.

So my contribution essentially transforms raw OSINT discovery into structured, explainable and privacy-scoped identity evidence rather than simply returning a list of search results."

PART XCVIII — 60-SECOND VERSION

Memorize this:

"I worked mainly on DigiZafe's OSINT discovery and identity-correlation pipeline. We start from a verified Identity Anchor and only use eligible identifiers or user-confirmed aliases. Maigret and OSINTgram discover candidate profiles, which are stored as CandidateProfiles with canonical URLs and provenance.

The key design principle is that discovery is not ownership. The Identity Match Engine therefore evaluates the candidate using deterministic

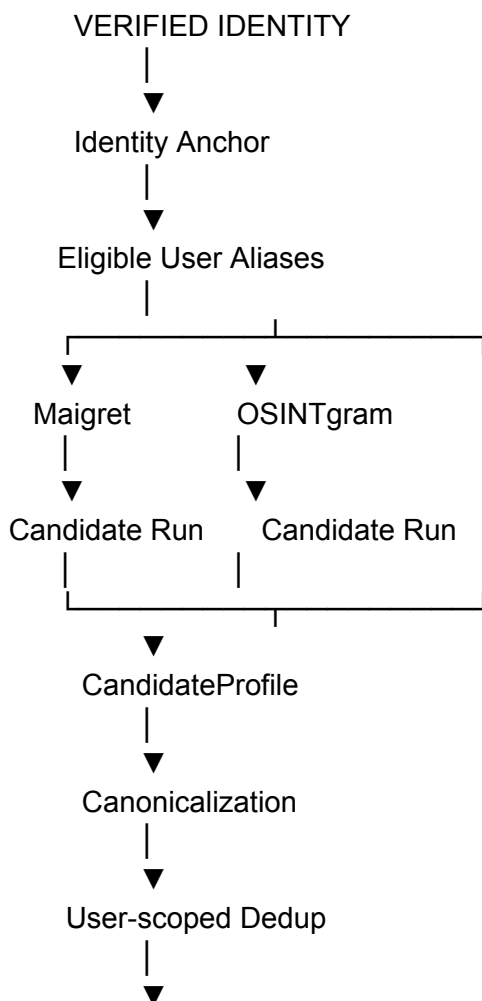
evidence rules, source reliability, independent evidence groups, collision risk, negative evidence and contradiction handling.

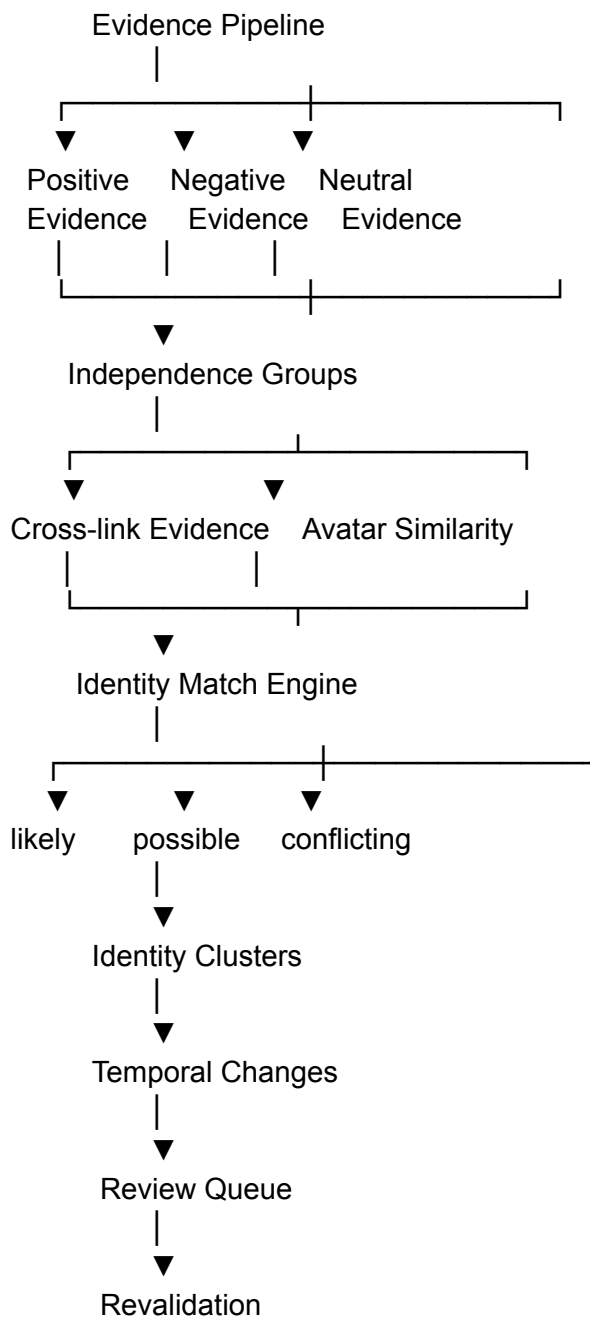
We later extended this with cross-link evidence, non-biometric avatar similarity and conservative identity clustering. Sprint 20 introduced connector orchestration with freshness, budgets, health and concurrency controls, while Sprint 21 added temporal change detection and a human review queue.

The goal is to convert noisy OSINT results into explainable, user-scoped identity evidence without automatically claiming that a discovered profile belongs to the user."

PART XCIX — YOUR MOST IMPORTANT DIAGRAM

If you can draw only one diagram in the viva, draw this:





PART C — THE 10 RULES YOU SHOULD NEVER FORGET

● Rule 1

Discovered ≠ owned.

● Rule 2

Likely ≠ confirmed.

● Rule 3

More observations ≠ more independent evidence.

● Rule 4

Connector failure ≠ negative evidence.

● Rule 5

Avatar similarity ≠ face recognition.

● Rule 6

Cross-link ≠ ownership proof.

● Rule 7

Cluster membership ≠ confirmed identity.

● Rule 8

User decisions remain authoritative and separate from algorithmic assessment.

● Rule 9

Historical evidence must not silently become current evidence.

● Rule 10

OSINT is evidence, not truth.

These principles are the backbone of the Sprint 16–21 identity architecture.

FINAL MENTAL MODEL FOR MEMBER 3

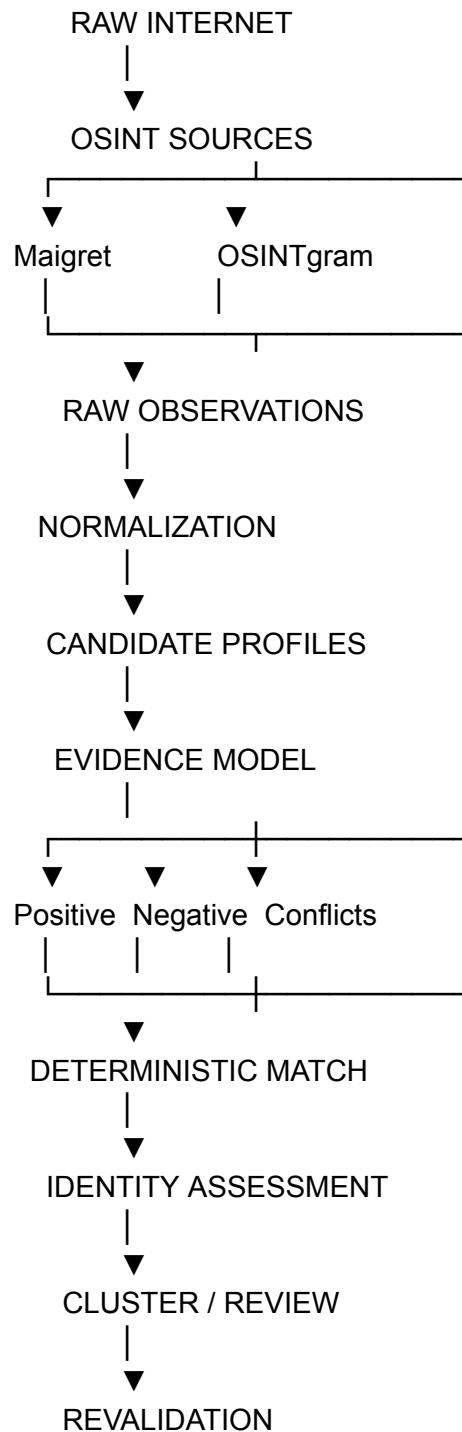
Member 2's job was:

"How do we act on confirmed exposure?"

Your job is:

"How do we determine what discovered information may actually relate to the user?"

So your entire contribution can be remembered as:



Your strongest one-line description for the viva is:

"I converted noisy multi-source OSINT discovery into deterministic, provenance-aware, user-scoped identity evidence while explicitly preserving uncertainty instead of treating discovery as identity proof."

That wording is closely aligned with the project's actual Sprint 16–21 architecture and is much stronger than simply saying **"I worked on OSINT."**